ĐẠI HỌC HOA SEN

TÔN TRỌNG SỰ KHÁC BIỆT

The NumPy Library

Faculty of Information Technology

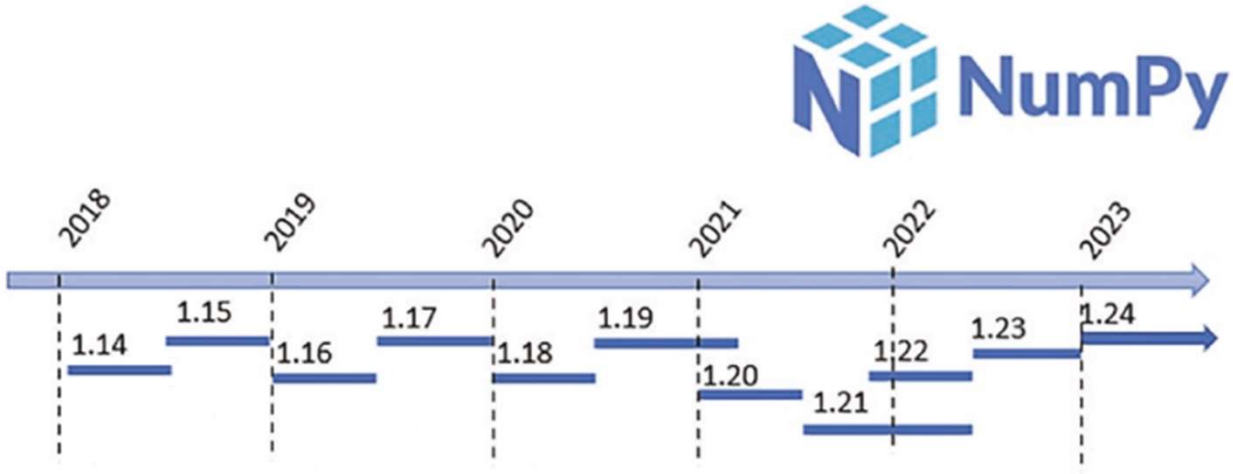
ĐẠI HỌC HOA SEN

Objectives

- ✓ NumPy
- ✓ ndarray
- ✓ Basic Operations
- ✓ Indexing, Slicing, and Iterating
- ✓ Array Manipulation
- ✓ Structured Arrays

NumPy

- NumPy is open source and licensed under BSD (Berkeley Software Distribution open-source operating system).
- There are many contributors who have expanded the potential of this library



ndarray

- Core Characteristics
  - ndarray = N-dimensional array - the primary NumPy object
  - Homogeneous: All elements must be the same type and size
  - Fixed size: Cannot grow or shrink (unlike Python lists)

```
import numpy as np
```

```
#Create an ndarray
arr = np.array([1,2,3,4]) # Python list
arr
```

```
array([1, 2, 3, 4])
```

- Important Properties
  - dtype: Specifies the data type of all elements in the array
  - shape: A tuple showing the size of each dimension (e.g., `(3,4)` for a 3×4 matrix)
  - axes: Another name for dimensions
  - rank: The number of axes/dimensions

- This homogeneity and fixed-size nature allows NumPy to perform operations much faster than Python lists, as memory layout is predictable and optimized for numerical computing.

- Attribute

```
#Create an ndarray
arr = np.array([1,2,3,4]) # Python list
arr
```

```
array([1, 2, 3, 4])
```

```
#dtype
arr.dtype
```

```
dtype('int64')
```

```
#axes attribute
arr.ndim
```

```
1
```

```
#array length attribute
arr.size
```

```
4
```

```
#shape attribute
arr.shape
```

```
(4,)
```

- two-dimensional array 2x2:

```
#array 2x2 - two dimensional
arr2 = np.array([[1.3, 2.4],[0.3, 4.1]])
arr2
```

```
array([[1.3, 2.4],
       [0.3, 4.1]])
```

```
arr2.dtype
```

```
dtype('float64')
```

```
arr2.ndim
```

```
2
```

```
arr2.size
```

```
4
```

```
arr2.shape
```

```
(2, 2)
```

List and Tuple

List

- A **list** is a mutable, ordered collection of items enclosed in square brackets [].
- **Characteristics:**
  - **Mutable**: Can be changed after creation (add, remove, modify items)
  - **Ordered**: Items maintain their position
  - **Dynamic**: Can grow or shrink in size
  - **Memory**: Uses more memory than tuples
  - **Performance**: Slower than tuples for iteration

Tuple

- A **tuple** is an immutable, ordered collection of items enclosed in parentheses ()
- **Characteristics:**
  - **Immutable**: Cannot be changed after creation
  - **Ordered**: Items maintain their position
  - **Fixed size**: Size cannot be changed
  - **Memory**: Uses less memory than lists
  - **Performance**: Faster than lists for iteration

Key Differences

Aspect	List	Tuple
Syntax	Square brackets []	Parentheses ()
Mutability	Mutable (can change)	Immutable (cannot change)
Speed	Slower	Faster
Memory	Uses more memory	Uses less memory
Use Case	Dynamic data that changes	Fixed data that shouldn't change

```python
# Creating a list
fruits = ["apple", "banana", "orange"]
print(fruits)  # Output: ['apple', 'banana', 'orange']

# Modifying list (mutable)
fruits[1] = "grape"
print(fruits)  # Output: ['apple', 'grape', 'orange']

# Adding items
fruits.append("mango")
print(fruits)  # Output: ['apple', 'grape', 'orange', 'mango']

# Removing items
fruits.remove("apple")
print(fruits)  # Output: ['grape', 'orange', 'mango']
```

```python
# Creating a tuple
coordinates = (10, 20, 30)
print(coordinates)  # Output: (10, 20, 30)

# Accessing items (immutable)
print(coordinates[1])  # Output: 20

# This would cause an error (immutable):
# coordinates[1] = 25  # TypeError: 'tuple' object does not support item assignment

# Tuples can contain different data types
person = ("John", 25, "Engineer")
print(person)  # Output: ('John', 25, 'Engineer')
```

Table 3-1. *Data Types Supported by NumPy*

Data Type	Description
`bool_`	Boolean (`true` or `false`) stored as a byte
`int_`	Signed integer type (same as C `long` and Python `int`; normally either `int64` or `int32` depending on the platform)
`intc`	Signed integer type, identical to C `int` (normally `int32` or `int64`)
`intp`	Integer used for indexing (same as C `size_t`; normally either `int32` or `int64`)
`int8`	Alias for the signed integer type with 8 bits (–128 to 127)
`int16`	Alias for the signed integer type with 16 bits (–32768 to 32767)
`int32`	Alias for the signed integer type with 32 bits (–2147483648 to 2147483647)
`int64`	Alias for the signed integer type with 64 bits (–9223372036854775808 to 9223372036854775807)
`uint8`	Alias for the unsigned integer type with 8 bits (0 to 255)
`uint16`	Alias for the unsigned integer type with 16 bits (0 to 65535)
`uint32`	Alias for the unsigned integer type with 32 bits (0 to 4294967295)
`uint64`	Alias for the unsigned integer type with 64 bits (0 to 18446744073709551615)
`float_`	Shorthand for `float64`
`float16`	Half precision float: sign bit, 5-bit exponent, 10-bit mantissa
`float32`	Single precision float: sign bit, 8-bit exponent, 23-bit mantissa
`float64`	Double precision float: sign bit, 11-bit exponent, 52-bit mantissa
`complex_`	Shorthand for `complex128`
`complex64`	Complex number, represented by two 32-bit floats (real and imaginary components)
`complex128`	Complex number, represented by two 64-bit floats (real and imaginary components)

The dtype option

- The array() function requires specifying a data type (dtype) for ndarray objects, which defines the type of data in each array item.
- By default, it automatically selects the most appropriate dtype based on the input values, but you can also explicitly define the dtype using the dtype option as an argument.

```
arr3 = np.array([[1, 2, 3],[4, 5, 6]], dtype=complex)
```

```
arr3
```

```
array([[1.+0.j, 2.+0.j, 3.+0.j],
       [4.+0.j, 5.+0.j, 6.+0.j]])
```

Intrinsic Creation of an Array

- The NumPy library provides a set of functions that generate ndarrays with initial content, created with different values depending on the function.

```
#The zeros() function: creates a full array of zeros with
#dimensions defined by the shape of the argument.
arr4 = np.zeros((3, 3))
```

```
arr4
```

```
array([[0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.]])
```

```
#the ones() function creates an array full of ones
arr5 = np.ones((3,3))
```

```
arr5
```

```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

- numpy.arange([start,] stop[, step,][, dtype])
  - **start (optional):** The beginning of the interval (inclusive). The default is 0.
  - **stop (required):** The end of the interval (exclusive, this value is *not* included in the array).
  - **step (optional):** The spacing or difference between consecutive values. The default is 1. A negative step can be used to create a descending sequence.
  - **dtype (optional):** The data type of the output array elements. If not specified, the type is inferred from the other arguments.

```
#to generate a sequence of values between 0 and 10
np.arange(0, 10)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
#specify two arguments: the first is the starting value
#and the second is the final value.
np.arange(4, 10)
```

```
array([4, 5, 6, 7, 8, 9])
```

```
#step 3
np.arange(0, 12, 3)
```

```
array([0, 3, 6, 9])
```

```
# step 0.6
np.arange(0, 6, 0.6)
```

```
array([0. , 0.6, 1.2, 1.8, 2.4, 3. , 3.6, 4.2, 4.8, 5.4])
```

```
#To generate two-dimensional arrays, you can still continue to use
#the arange() function but combined with the reshape() function.
np.arange(0, 12).reshape(3, 4)
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
# the third argument, instead of specifying the distance between one element
#and the next, defines the number of elements into which you want
#the interval to be split.
np.linspace(0,10,5)
```

```
array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

```
np.random.random(3)
```

```
array([0.61217785, 0.01709539, 0.13309417])
```

```
np.random.random((3,3))
```

```
array([[0.55733869, 0.31409439, 0.40710167],
       [0.37295229, 0.1314394 , 0.18386402],
       [0.80892422, 0.06991653, 0.39628544]])
```

Basic Operations

Arithmetic Operators

- **Addition:** Performed using the + operator or the np.add() function.
- **Subtraction:** Performed using the - operator or the np.subtract() function.
- **Multiplication:** Performed using the * operator or the np.multiply() function. Note that * is element-wise multiplication, not matrix multiplication.
- **Division:** Performed using the / operator or the np.divide() function.
- **Power/Exponentiation:** Use the ** operator or the np.power() function.
- **Modulus:** Use the % operator or the np.mod() function to find the remainder.

- Example

```
arrA
```

```
array([0, 1, 2, 3])
```

```
arrA+4
```

```
array([4, 5, 6, 7])
```

```
arrA*2
```

```
array([0, 2, 4, 6])
```

```
arrB = np.arange(4,8)
```

```
arrB
```

```
array([4, 5, 6, 7])
```

```
arrA + arrB
```

```
array([-4, -4, -4, -4])
```

```
arrA - arrB
```

```
array([-4, -4, -4, -4])
```

```
arrA * arrB
```

```
array([ 0,  5, 12, 21])
```

```
#multiply the array by the sine or the square root of the elements of array arrB.
arrA * np.sin(arrB)
```

```
array([-0.        , -0.95892427, -0.558831  ,  1.9709598 ])
```

```
arrA * np.sqrt(arrB)
```

```
array([0.        , 2.23606798, 4.89897949, 7.93725393])
```

- Example: the multidimensional case

```
arrC = np.arange(0, 9).reshape(3, 3)
```

```
arrC
```

```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

```
arrD = np.ones((3, 3))
```

```
arrD
```

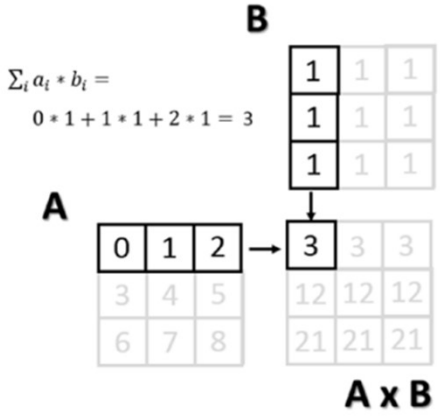
```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

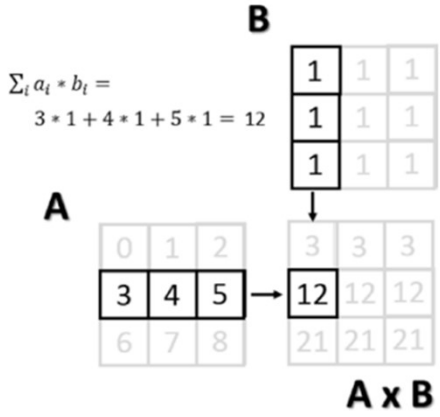
```
arrC * arrD
```

```
array([[0., 1., 2.],
       [3., 4., 5.],
       [6., 7., 8.]])
```

The Matrix Product

- The choice of operating element-wise is a peculiar aspect of the NumPy library. In fact, in many other tools for data analysis, the * operator is understood as a *matrix product* when it is applied to two matrices. Using NumPy, this kind of product is instead indicated by the dot() function. This operation is not element-wise.






```
np.dot(arrC, arrD)

array([[ 3.,  3.,  3.],
       [12., 12., 12.],
       [21., 21., 21.]])
```

```
np.dot(arrD, arrC)

array([[ 9., 12., 15.],
       [ 9., 12., 15.],
       [ 9., 12., 15.]])
```

Increment and Decrement Operators

- use operators such as += and –=

```
arrX = np.arange(4)
```

```
arrX
```

```
array([0, 1, 2, 3])
```

```
arrX += 1
```

```
arrX
```

```
array([1, 2, 3, 4])
```

```
arrX -= 1
```

```
arrX
```

```
array([0, 1, 2, 3])
```

```
arrX += 4
```

```
arrX
```

```
array([4, 5, 6, 7])
```

```
arrX *= 2
```

```
arrX
```

```
array([ 8, 10, 12, 14])
```

Universal Functions (ufunc)

- A universal function, generally called *ufunc*, is a function operating on an array in an element-by-element fashion. This means that it acts individually on each single element of the input array to generate a corresponding result in a new output array.

```
arrX = np.arange(1,5)
```

```
arrX
```

```
array([1, 2, 3, 4])
```

```
np.sqrt(arrX)
```

```
array([1.        , 1.41421356, 1.73205081, 2.        ])
```

```
np.log(arrX)
```

```
array([0.        , 0.69314718, 1.09861229, 1.38629436])
```

```
np.sin(arrX)
```

```
array([ 0.84147098,  0.90929743,  0.14112001, -0.7568025 ])
```

Aggregate Functions

- Aggregate functions perform an operation on a set of values, an array for example, and produce a single result. Therefore, the sum of all the elements in an array is an aggregate function.
- sum(), max(), min(), mean(), std()

```
arrX = np.array([3.3, 4.5, 1.2, 5.7, 0.3])
```

```
arrX
```

```
array([3.3, 4.5, 1.2, 5.7, 0.3])
```

```
arrX.sum()
```

```
np.float64(15.0)
```

```
arrX.max()
```

```
np.float64(5.7)
```

```
arrX.min()
```

```
np.float64(0.3)
```

```
arrX.mean()
```

```
np.float64(3.0)
```

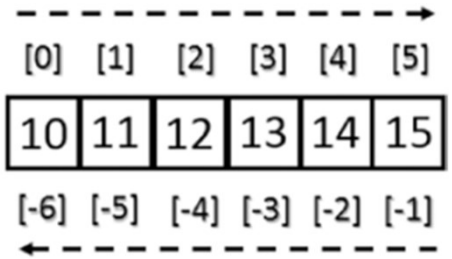
```
arrX.std()
```

```
np.float64(2.0079840636817816)
```

Indexing, Slicing, and Iterating

Indexing

- Array indexing always uses square brackets ([ ]) to index the elements of the array so that the elements can then be referred individually for various uses, such as extracting a value, selecting items, or even assigning a new value.




```
arrX = np.arange(10, 16)
```

```
arrX
```

```
array([10, 11, 12, 13, 14, 15])
```

```
arrX[3]
```

```
np.int64(13)
```

```
arrX[-1]
```

```
np.int64(15)
```

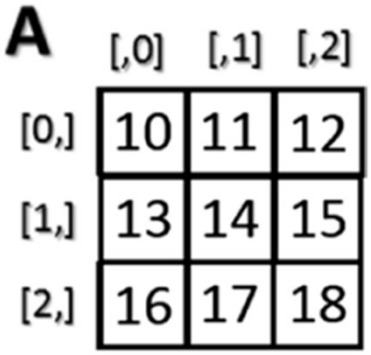
```
arrX[-6]
```

```
np.int64(10)
```

```
arrX[[1, 3, 4]]
```

```
array([11, 13, 14])
```

- Indexing a two-dimensional array




```
arrY = np.arange(10, 19).reshape((3, 3))
```

```
arrY
```

```
array([[10, 11, 12],
       [13, 14, 15],
       [16, 17, 18]])
```

```
arrY[1,2]
```

```
np.int64(15)
```

Slicing

- *Slicing* allows you to extract portions of an array to generate new arrays. When you use the Python lists to slice arrays, the resulting arrays are copies, but in NumPy, the arrays are views of the same underlying buffer.
- Depending on the portion of the array that you want to extract (or view), you must use the slice syntax; that is, you use a sequence of numbers separated by colons (:) within square brackets.

```
arrX = np.arange(10, 16)
```

```
arrX
```

```
array([10, 11, 12, 13, 14, 15])
```

```
#from the second to the sixth element
arrX[1:5]
```

```
array([11, 12, 13, 14])
```

```
#to extract an item from the previous portion and skip a specific
#number of following items, then extract the next and skip again, you can use
#a third number that defines the gap in the sequence of the elements.
arrX[1:5:2]
```

```
array([11, 13])
```

- To better understand the slice syntax, you also should look at cases where you do not use explicit numerical values. If you omit the first number, NumPy implicitly interprets this number as 0 (i.e., the initial element of the array). If you omit the second number, this will be interpreted as the maximum index of the array; and if you omit the last number, this will be interpreted as 1. All the elements will be considered without intervals.

```
array([10, 11, 12, 13, 14, 15])
```

```
arrX[::2]
```

```
array([10, 12, 14])
```

```
arrX[:5:2]
```

```
array([10, 12, 14])
```

```
arrX[:5:]
```

```
array([10, 11, 12, 13, 14])
```

- In the case of a two-dimensional array, the slicing syntax still applies, but it is separately defined for the rows and columns.

```
arrY = np.arange(10, 19).reshape((3, 3))
```

```
arrY
```

```
array([[10, 11, 12],
       [13, 14, 15],
       [16, 17, 18]])
```

```
#the first row (index = 0)
arrY[0,:]
```

```
array([10, 11, 12])
```

```
#the first column (index = 0)
arrY[:,0]
```

```
array([10, 13, 16])
```

```
#if you want to extract a smaller matrix, you need to explicitly
#define all intervals with indexes that define them.
arrY[0:2, 0:2]
```

```
array([[10, 11],
       [13, 14]])
```

```
#If the indexes of the rows or columns to be extracted are not contiguous,
#you can specify an array of indexes.
arrY[[0,2], 0:2]
```

```
array([[10, 11],
       [16, 17]])
```

Iterating an Array

- use the for construct

```
for i in arrX:
  print(i)
```

```
10
11
12
13
14
15
```

```
for i in arrY:
    print(i)
```

```
[10 11 12]
[13 14 15]
[16 17 18]
```

Conditions and Boolean Arrays

- To selectively extract the elements in an array is to use the conditions and Boolean operators.

```
arrY = np.random.random((4, 4))
```

```
arrY
```

```
array([[0.65157879, 0.2536621 , 0.88878558, 0.37948334],
       [0.59358011, 0.90809438, 0.20751144, 0.67217345],
       [0.25189902, 0.49021204, 0.82977324, 0.38718072],
       [0.57157502, 0.25367404, 0.23627818, 0.08865956]])
```

```
#a Boolean array containing true values in the positions in which
#the condition is satisfied
arrY < .5
```

```
array([[False,  True, False,  True],
       [False, False,  True, False],
       [ True,  True, False,  True],
       [False,  True,  True,  True]])
```

```
#extract all elements smaller than 0.5
arrY[arrY < .5]
```

```
array([0.2536621 , 0.37948334, 0.20751144, 0.25189902, 0.49021204,
       0.38718072, 0.25367404, 0.23627818, 0.08865956])
```

Shape Manipulation

- The reshape() function: convert a one-dimensional array into a matrix

```
arrA = np.random.random(12)
```

```
arrA
```

```
array([0.62920396, 0.45277158, 0.99153186, 0.58161725, 0.12052886,
       0.44476363, 0.95686226, 0.66787578, 0.89381181, 0.83481152,
       0.58893331, 0.19985583])
```

```
arrB = arrA.reshape(3,4)
```

```
arrB
```

```
array([[0.62920396, 0.45277158, 0.99153186, 0.58161725],
       [0.12052886, 0.44476363, 0.95686226, 0.66787578],
       [0.89381181, 0.83481152, 0.58893331, 0.19985583]])
```

- The shape attribute: to modify the object by modifying the shape, you have to assign a tuple containing the new dimensions directly to its shape attribute.

```
arrA.shape = (3,4)
arrA
```

```
array([[0.62920396, 0.45277158, 0.99153186, 0.58161725],
       [0.12052886, 0.44476363, 0.95686226, 0.66787578],
       [0.89381181, 0.83481152, 0.58893331, 0.19985583]])
```

- The ravel() function: convert a two-dimensional array into a one-dimensional array

```
arrA = arrA.ravel()
arrA
```

```
array([0.62920396, 0.45277158, 0.99153186, 0.58161725, 0.12052886,
       0.44476363, 0.95686226, 0.66787578, 0.89381181, 0.83481152,
       0.58893331, 0.19985583])
```

- The transpose() function: inverting the columns with the rows

```
arrY
```

```
array([[0.65157879, 0.2536621 , 0.88878558, 0.37948334],
       [0.59358011, 0.90809438, 0.20751144, 0.67217345],
       [0.25189902, 0.49021204, 0.82977324, 0.38718072],
       [0.57157502, 0.25367404, 0.23627818, 0.08865956]])
```

```
arrY.transpose()
```

```
array([[0.65157879, 0.59358011, 0.25189902, 0.57157502],
       [0.2536621 , 0.90809438, 0.49021204, 0.25367404],
       [0.88878558, 0.20751144, 0.82977324, 0.23627818],
       [0.37948334, 0.67217345, 0.38718072, 0.08865956]])
```

Array Manipulation

Joining Arrays

- Merge multiple arrays to form a new one that contains all of the arrays. NumPy uses the concept of *stacking*, providing a number of functions in this regard.
- The vstack() function: combines the second array as new rows of the first array. In this case, the array grows in the vertical direction.
- The hstack() function performs horizontal stacking; that is, the second array is added to the columns of the first array.

```
arrA = np.ones((3, 3))
arrB = np.zeros((3, 3))
```

```
np.vstack((arrA, arrB))
```

```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.],
       [0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.]])
```

```
np.hstack((arrA, arrB))
```

```
array([[1., 1., 1., 0., 0., 0.],
       [1., 1., 1., 0., 0., 0.],
       [1., 1., 1., 0., 0., 0.]])
```

- Two other functions performing stacking between multiple arrays are column_stack() and vstack().

```
arr1 = np.array([0, 1, 2])
arr2 = np.array([3, 4, 5])
arr3 = np.array([6, 7, 8])
```

```
np.column_stack((arr1, arr2, arr3))
```

```
array([[0, 3, 6],
       [1, 4, 7],
       [2, 5, 8]])
```

```
#`row_stack` alias is deprecated. Use `np.vstack` directly.
np.vstack((arr1, arr2, arr3))
```

```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

Splitting Arrays

- To divide an array into several parts.
- In NumPy: horizontally with the hsplit() function and vertically with the vsplit() function.

```
arrY1 = np.arange(16).reshape((4, 4))
```

```
arrY1
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])
```

```
[arrY2, arrY3]= np.hsplit(arrY1, 2)
```

```
arrY2
```

```
array([[ 0,  1],
       [ 4,  5],
       [ 8,  9],
       [12, 13]])
```

```
arrY3
```

```
array([[ 2,  3],
       [ 6,  7],
       [10, 11],
       [14, 15]])
```

```
[arrY4, arrY5]= np.vsplit(arrY1, 2)
```

```
arrY4
```

```
array([[0, 1, 2, 3],
       [4, 5, 6, 7]])
```

```
arrY5
```

```
array([[ 8,  9, 10, 11],
       [12, 13, 14, 15]])
```

- A more complex command is the split() function, which allows you to split the array into nonsymmetrical parts.
- Passing the array as an argument, you also have to specify the indexes of the parts to be divided. If you use the **axis = 1** option, then the indexes will be **columns**; if instead the option is **axis = 0**, then they will be **row** indexes.

```
arrY = np.arange(16).reshape((4, 4))
```

```
arrY
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])
```

```
#Column
[arrC1,arrC2,arrC3] = np.split(arrY,[1,3],axis=1)
```

```
arrC1
```

```
array([[0],
       [3],
       [6]])
```

```
arrC2
```

```
array([[ 1,  2],
       [ 5,  6],
       [ 9, 10],
       [13, 14]])
```

```
arrC3
```

```
array([[ 3],
       [ 7],
       [11],
       [15]])
```

```
#Row
[arrR1,arrR2,arrR3] = np.split(arrY,[1,3],axis=0)
```

```
arrR1
```

```
array([[0, 1, 2, 3]])
```

```
arrR2
```

```
array([[ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
arrR3
```

```
array([[12, 13, 14, 15]])
```

Structured Arrays

- This type of array contains structs or records instead of individual items
- For example, you can create a simple array of structs as items. Thanks to the dtype option, you can specify a list of comma-separated specifiers to indicate the elements that will constitute the struct, along with data type and order.
  - bytes b1
  - int i1, i2, i4, i8
  - unsigned ints u1, u2, u4, u8
  - floats f2, f4, f8
  - complex c8, c16
  - fixed length strings S<n>

- Example

```
structured = np.array([(1, 'First', 0.5, 1+2j), (2, 'Second', 1.3, 2-2j),
 (3, 'Third', 0.8, 1+3j)], dtype='i2, S6, f4, c8')
```

```
structured
```

```
array([(1, b'First', 0.5, 1.+2.j), (2, b'Second', 1.3, 2.-2.j),
       (3, b'Third', 0.8, 1.+3.j)],
      dtype=[('f0', '<i2'), ('f1', 'S6'), ('f2', '<f4'), ('f3', '<c8')])
```

f0, f1, f2,f3: field name
Can write:
'f0': 'id'
'f1': 'position'
'f2': 'value'
'f3': 'complex'

```
#use the data type explicitly by specifying int8, uint8, float16, complex64, and so forth.
structured = np.array([(1, 'First', 0.5, 1+2j),(2, 'Second', 1.3,2-2j),
 (3, 'Third', 0.8, 1+3j)],dtype='int16, S6, float32, complex64')
```

```
array([(1, b'First', 0.5, 1.+2.j), (2, b'Second', 1.3, 2.-2.j),
       (3, b'Third', 0.8, 1.+3.j)],
      dtype=[('f0', '<i2'), ('f1', 'S6'), ('f2', '<f4'), ('f3', '<c8')])
```

- Writing the appropriate reference index, you obtain the corresponding row, which contains the struct.

```
structured[1]
```

```
np.void((2, b'Second', 1.3, 2.-2.j), dtype=[('f0', '<i2'), ('f1', 'S6'), ('f2', '<f4'), ('f3', '<c8')])
```

- The names that are assigned automatically to each item of the struct can be considered the names of the columns of the array. Using them as a structured index, you can refer to all the elements of the same type, or of the same column.

```
structured['f1']
```

```
array([b'First', b'Second', b'Third'], dtype='|S6')
```

- Redefining the tuples of names assigned to the dtype attribute of the structured array.

```
structured.dtype.names = ('id','order','value','complex')
```

```
structured['order']
```

```
array([b'First', b'Second', b'Third'], dtype='|S6')
```


Q&A